STM32F446xx SPI Driver Documentation

1. Introduction

The SPI (Serial Peripheral Interface) driver provides a simple bare-metal interface for configuring and communicating with SPI peripherals on the STM32F446xx microcontroller. The driver supports Master mode operation and basic full-duplex data transmission.

The driver is divided into two files:

- **stm32f446xx_spi_driver.h** – Contains data structures, macros, and function declarations.
- **stm32f446xx_spi_driver.c** – Contains the implementation of SPI initialization and communication functions.

---

2. Header File (stm32f446xx_spi_driver.h)

2.1 SPI Configuration Structure

```
typedef struct {
  uint8_t SPI_DeviceMode;
  uint8_t SPI_BusConfig;
  uint8_t SPI_SclkSpeed;
  uint8_t SPI_DFF;
  uint8_t SPI_CPOL;
  uint8_t SPI_CPHA;
  uint8_t SPI_SSM;
} SPI_Config_t;
```

Purpose

Stores all configuration parameters required for SPI initialization.

Members

Member	Description
SPI_DeviceMode	Master or Slave mode
SPI_BusConfig	Full duplex configuration

Member	Description
SPI_SclkSpeed	Clock prescaler value
SPI_DFF	Data frame format (8-bit/16-bit)
SPI_CPOL	Clock polarity
SPI_CPHA	Clock phase
SPI_SSM	Software slave management

2.2 SPI Handle Structure

```
typedef struct {
  SPI_RegDef_t *pSPIx;
  SPI_Config_t SPI_Config;
} SPI_Handle_t;
```

Purpose

Combines the SPI peripheral base address with its configuration.

2.3 Configuration Macros

```
#define SPI_DEVICE_MODE_MASTER    1
#define SPI_BUS_CONFIG_FULL_DUPLEX 1
#define SPI_DFF_8BITS          0
#define SPI_CPOL_LOW           0
#define SPI_CPHA_LOW           0
#define SPI_SSM_EN             1
#define SPI_SCLK_SPEED_DIV32     4
```

Purpose

Provides readable names for commonly used SPI settings.

---

3. Driver Functions

3.1 SPI_PeriClockControl()

Prototype

void SPI_PeriClockControl(SPI_RegDef_t *pSPIx, uint8_t EnOrDi);

Purpose

Enables or disables the SPI peripheral clock.

Parameters

Parameter	Description
pSPIx	SPI peripheral pointer
EnOrDi	ENABLE or DISABLE

Working

If ENABLE:

RCC->APB2ENR |= (1<<12);

This enables the SPI1 peripheral clock.

If DISABLE:

RCC->APB2ENR &= ~(1<<12);

This disables the SPI1 peripheral clock.

3.2 SPI_Init()

Prototype

void SPI_Init(SPI_Handle_t *pSPIHandle);

Purpose

Initializes the SPI peripheral according to the configuration structure.

Configuration Steps

1. Configure Master mode.
2. Configure Full Duplex communication.
3. Configure baud rate.
4. Configure data frame format.

5. Configure Clock Polarity (CPOL).
6. Configure Clock Phase (CPHA).
7. Enable Software Slave Management.
8. Set SSI bit to avoid MODF fault.

Register Used

SPI_CR1

3.3 SPI_SendData()

Prototype

```
void SPI_SendData(
  SPI_RegDef_t *pSPIx,
  uint8_t *pTxBuffer,
  uint32_t Len
);
```

Purpose

Transmits multiple bytes through SPI.

Working

For every byte:

1. Wait until TXE flag becomes SET.
2. Write one byte into DR register.
3. Increment buffer pointer.
4. Repeat until all bytes are transmitted.

Pseudo flow:

```
while(length > 0)
{
  wait TXE
  DR = data
  next byte
```

}

3.4 SPI_SendReceiveByte()

Prototype

```
uint8_t SPI_SendReceiveByte(
  SPI_RegDef_t *pSPIx,
  uint8_t txByte
);
```

Purpose

Performs simultaneous transmission and reception of one byte.

Working

Step 1

Wait until TXE is SET.

Step 2

Write transmit byte into DR.

Step 3

Wait until RXNE is SET.

Step 4

Read received byte from DR and return it.

Flow:

Wait TXE

↓

Write DR

↓

Wait RXNE

↓

Read DR

↓

Return received byte

3.5 SPI_PeripheralControl()

Prototype

```
void SPI_PeripheralControl(
  SPI_RegDef_t *pSPIx,
  uint8_t EnOrDi
);
```

Purpose

Enables or disables the SPI peripheral.

Enable

CR1 |= (1<<6)

Sets SPE bit.

Disable

CR1 &= ~(1<<6)

Clears SPE bit.

---

4. SPI Registers Used

SPI_CR1

Controls:

- Master/Slave selection
- Clock polarity
- Clock phase
- Baud rate
- Software slave management
- SPI enable

Important bits:

Bit	Function
6	SPE (SPI Enable)
2	MSTR
1	CPOL
0	CPHA
9	SSM
8	SSI
5:3	Baud Rate

SPI_SR

Status register used during communication.

Important bits:

Bit	Function
TXE	Transmit buffer empty
RXNE	Receive buffer not empty
BSY	SPI busy flag

SPI_DR

Data register used for transmission and reception.

Writing:

DR = transmit_data;

Reading:

received_data = DR;

---

5. Driver Features

- Bare-metal implementation

- Master mode support
- Full duplex communication
- 8-bit data frame
- Software Slave Management (SSM)
- Configurable baud rate
- Blocking transmit API
- Simultaneous transmit/receive API

---

6. Advantages

- Simple implementation
- Easy to understand for embedded beginners
- Modular driver structure
- Reusable for SPI-based sensors and peripherals
- No dependency on STM32 HAL libraries

---

7. Limitations

- Supports only SPI1 clock control.
- Only blocking mode communication.
- No interrupt support.
- No DMA support.
- No receive-only API.
- No error handling or timeout mechanism.

---

8. Conclusion

This SPI driver provides a basic bare-metal implementation for STM32F446xx microcontrollers. It initializes the SPI peripheral, enables/disables the peripheral clock, performs data transmission, and supports simultaneous transmit/receive operations. The modular design makes it suitable for interfacing with SPI devices such as LCDs,

sensors, EEPROMs, and Arduino boards while serving as a foundation for more advanced SPI driver development.